
Your Language Model is Its Own Critic: Reinforcement Learning with Value Estimation from Actor’s Internal States

Yunho Choi*

Graduate School of Data Science
Seoul National University
dbsgh7177@snu.ac.kr

Jongwon Lim*

Graduate School of Data Science
Seoul National University
elijah0430@snu.ac.kr

Woojin Ahn

Computer Science and Engineering
Seoul National University
awj1204@snu.ac.kr

Minjae Oh

Graduate School of Data Science
Seoul National University
kosair@snu.ac.kr

Jeonghoon Shim

Graduate School of Data Science
Seoul National University
jhshim98@snu.ac.kr

Yohan Jo[†]

Graduate School of Data Science
Seoul National University
yohan.jo@snu.ac.kr

Abstract

Reinforcement learning with verifiable rewards (RLVR) for Large Reasoning Models hinges on baseline estimation for variance reduction, but existing approaches pay a heavy price: PPO requires a policy-model scale critic, while GRPO needs multiple rollouts per prompt to keep its empirical group mean stable. We introduce **POISE** (Policy Optimization with Internal State Value Estimation), which obtains a baseline at negligible cost by using the policy model’s internal signals already computed during the policy forward pass. A lightweight probe predicts the expected verifiable reward from the hidden states of the prompt and generated trajectory, as well as token-entropy statistics, and is trained online alongside the policy. To preserve gradient unbiasedness despite using trajectory-conditioned features, we introduce a cross-rollout construction that predicts each rollout’s value from an independent rollout’s internal states. Because POISE estimates prompt value using only a single rollout, it enables higher prompt diversity for a fixed compute budget during training. This reduces gradient variance for more stable learning and also eliminates the compute overhead of sampling costs for detecting zero-advantage prompts. On Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B across math reasoning benchmarks, POISE matches DAPO while requiring less compute. Moreover, its value estimator shows similar performance to a separate LLM-scale value model and generalizes to various verifiable tasks. By leveraging the model’s own internal representations, POISE enables more stable and efficient policy optimization.¹

¹We will release the code upon the publication of the paper.

*Equal contribution.

[†]Corresponding author.

1 Introduction

Large language models (LLMs) have recently shown remarkable improvements on complex reasoning tasks by generating long chains of thought before committing to a final answer [12, 36]. A central driver of this progress has been reinforcement learning with verifiable rewards (RLVR), which optimizes the model using outcome-level rewards [9, 14]. To reduce reward variance and the resulting training instability, a baseline is subtracted from the reward to form an advantage—a measure of how much better a given response is relative to what the model would typically achieve. Obtaining a reliable baseline is therefore central to stable and efficient RLVR.

Yet existing approaches pay a significant computational price to do so. Proximal Policy Optimization (PPO) [24] trains an LLM-scale critic with the policy to produce per-token baseline values; critic must process the full generated sequence at every update, roughly doubling memory consumption and increasing the optimization complexity. Group Relative Policy Optimization (GRPO) [25] sidesteps the critic by estimating a per-prompt baseline as the mean reward over a group of sampled rollouts, but this trades parameters for samples: a reliable estimation of the baseline requires multiple rollouts per prompt, which under a fixed compute budget reduces in-batch prompt diversity and, in turn, inflates the variance of gradient estimates [7] (see § 2.3). Substantial compute is also spent on uninformative prompts, for which all rollouts receive identical rewards and therefore yield zero advantage [37]. As reasoning trajectories grow longer, both costs compound and consume compute that could otherwise be used for learning. Underlying both approaches is the same bottleneck: producing a baseline demands extra resources. This motivates the central question of our work: *Can an effective baseline be extracted from the computations already performed during policy training?*

We suggest that a promising answer to this question is to leverage the information encoded in the policy model’s own internal representations to estimate the baseline. This hypothesis is grounded in a growing body of work showing that hidden states of LLMs and LRMs encode outcome-relevant information such as perceived difficulty, capability boundaries, and answer correctness, which can serve as a highly informative proxy for expected rewards. Yet these signals have been treated purely as diagnostic tools at inference time, leaving their potential to inform training entirely unexplored.

In this paper, we propose **POISE** (Policy Optimization with Internal State Value Estimation), a reinforcement learning algorithm that turns the model’s internal states into a value model. Concretely, we train a lightweight probe that predicts the value $V^\pi(x) = \mathbb{E}_{y \sim \pi(\cdot|x)}[R(x, y)]$ from internal signals collected at two levels. The first is a *prompt-level* feature, extracted from hidden states at the final prompt tokens before generation begins, which captures how the model represents the prompt and its anticipated difficulty under the current policy. The second is a *trajectory-level* feature, comprising hidden states taken when the model’s reasoning ends together with token-level entropy. Because using rollout-dependent signals in the baseline biases the gradient estimator, we pair each rollouts with a second, independent rollout from the same prompt. The probe predicts the *paired* rollout’s value thereby making the value independent to the corresponding rollout. This cross-rollout architecture keeps the baseline conditionally independent of the action which otherwise introduce bias into the gradient estimator [29, 33], so the probe is driven to recover the policy’s expected reward $V^\pi(x)$ rather than to memorize trajectory-specific outcomes. Trained jointly with the policy on a sliding buffer of recent rollouts, our value estimator tracks the evolving policy with negligible overhead.

Our method offers several concrete advantages over existing approaches. Unlike PPO, the baseline is supplied by a lightweight value estimator rather than an LLM-scale critic. Compared to GRPO, our method requires only a pair of rollouts rather than a large group; the saved budget can be redirected to more distinct prompts per batch, improving training stability. Moreover, because the value estimator provides a lightweight continuous baseline for each rollout, POISE avoids the extra sampling needed to identify and discard degenerate zero-advantage prompt groups.

We validate these claims experimentally. POISE matches DAPO [37]: a state of the art, GRPO-based RL algorithm in LLM reasoning, with less compute. We also show that our lightweight value estimator performs similar to an LLM-scale value model in performance (Figure 1), despite relying only on signals already produced during the policy’s forward pass. Beyond these performance results, we analyze the estimator itself (§ 5), identifying which layers and signals contribute most to value prediction and tracking how the estimator evolves alongside the policy during training. Finally, we demonstrate that the estimator generalizes beyond mathematical reasoning, yielding consistent gains on coding, tool-calling, and instruction-following tasks.

Overall, we show that internal representations of reasoning models can move beyond their conventional use as diagnostic tools for reasoning behavior and serve as practical optimization signals for reinforcement learning. Without group-relative baselines or a separate critic model, our method provides a compute-efficient path toward stable and scalable RLVR for large reasoning models.

2 Preliminaries

2.1 Policy Gradient and Baseline Estimation

We formulate RLVR for LLM reasoning as a contextual bandit problem over prompt–response pairs [31]. Given a prompt $x \sim \mathcal{D}$ and a response $y \sim \pi_\theta(\cdot | x)$ sampled from the policy model, the objective is to maximize the expected verifiable reward $R(x, y)$,

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [R(x, y)]. \quad (1)$$

By the policy gradient theorem,

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [R(x, y) \nabla_\theta \log \pi_\theta(y | x)], \quad (2)$$

which yields the REINFORCE estimator [33]. In practice, this estimator is typically combined with a baseline $b(x)$ to reduce variance [28], giving the advantage $A(x, y) = R(x, y) - b(x)$ and the gradient estimator

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [(R(x, y) - b(x)) \nabla_\theta \log \pi_\theta(y | x)]. \quad (3)$$

The standard near-optimal choice for variance reduction is the value function [8, 32]

$$V^{\pi_\theta}(x) = \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [R(x, y)], \quad (4)$$

which is unknown and must be estimated in practice. PPO approximates $V^{\pi_\theta}(x)$ with a learned critic v_ϕ that is trained jointly with the policy, providing a direct parametric estimate of the value function. GRPO instead samples a group of G responses $\{y^{(1)}, \dots, y^{(G)}\}$ for the same prompt and uses their mean reward as an empirical prompt-level baseline,

$$b_{\text{GRPO}}(x) = \frac{1}{G} \sum_{j=1}^G R(x, y^{(j)}), \quad (5)$$

obtaining the baseline directly from on-policy rollouts.

2.2 Unbiasedness Condition for Baselines

Subtracting a baseline preserves the unbiasedness of the policy gradient only when the baseline term has zero expectation:

$$\mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [b(x) \nabla_\theta \log \pi_\theta(y | x)] = 0. \quad (6)$$

This condition holds when the baseline is conditionally independent of the sampled response y given the prompt x , in which case

$$\mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [b(x) \nabla_\theta \log \pi_\theta(y | x)] = b(x) \nabla_\theta \sum_y \pi_\theta(y | x) = 0. \quad (7)$$

Equivalently, a baseline may depend only on the prompt or on any quantity that is independent of the sampled response given the prompt. Violating this condition biases the gradient and can drive the policy to converge suboptimally. We therefore adopt a *cross-rollout* construction, where the baseline for a response is computed from another independent response, preserving Eq. (6).

2.3 Gradient Variance and Number of Prompts in the Batch

A baseline estimator that requires fewer rollouts per prompt can reallocate the same completion budget toward more distinct prompts in each batch. This section formalizes why such prompt diversity matters for policy optimization. We show that, under a fixed compute budget, allocating rollouts across more distinct prompts reduces the noise of the gradient estimate.

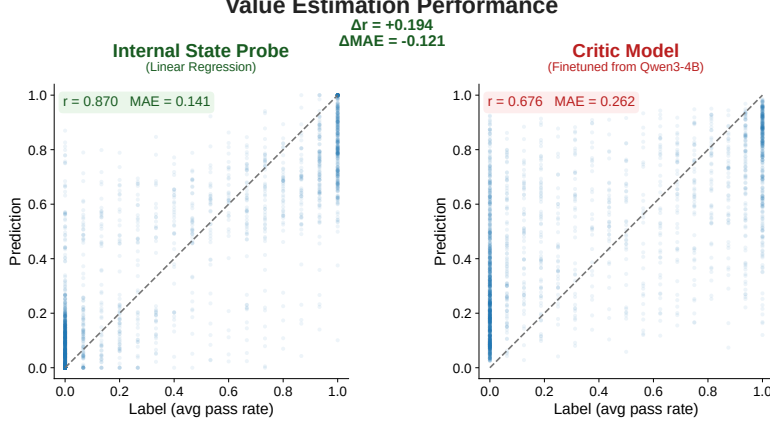


Figure 1: Comparing value prediction between our internal state probe and a separately trained critic model. Predictions are compared against empirical Avg@8 scores. Our probe achieves higher Pearson correlation (r) and lower MAE, indicating that the policy’s own internal representations provide an effective low-cost signal for value estimation.

Let B be the total number of completions in a training batch, with n distinct prompts and m completions each, so $B = n \cdot m$. For prompt $x^{(i)} \sim D$ and completion $y^{(ij)} \sim \pi_\theta(\cdot | x^{(i)})$, define the per-sample gradient:

$$Z(x, y) = \nabla_\theta \log \pi_\theta(y | x) (R(x, y) - b(x)) \quad (8)$$

where $b(x)$ is a baseline. The batch gradient estimator is:

$$\hat{g} = \frac{1}{n} \sum_{i=1}^n \frac{1}{m} \sum_{j=1}^m Z(x^{(i)}, y^{(ij)}). \quad (9)$$

Proposition 1 (Gradient variance decomposition). *Let Σ_w and Σ_b denote the within-prompt and between-prompt covariance matrices of Z . Both Σ_w and Σ_b are fixed properties of (D, π_θ, R) , independent of the allocation (n, m) . Then:*

$$\text{Cov}(\hat{g}) = \frac{1}{B} \Sigma_w + \frac{m}{B} \Sigma_b. \quad (10)$$

Proof. See § A.1. □

Corollary 1 (Optimal allocation). *For a fixed budget B and baseline $b(x)$, the variance of $\hat{g}$ is monotonically non-decreasing in m (in the Loewner order) and is minimized at $m = 1, n = B$.*

Proof. See § A.2. □

In other words, given the same total budget, using as many diverse prompts as possible is critical to stable learning (i.e., $m = 1$ or 2). Yet GRPO requires repeated sampling from the same prompt to estimate a faithful baseline $b(x)$. This motivates our method of estimating a reliable baseline with minimal sampling without training a separate value network as in PPO.

3 Policy Optimization with Internal State Value Estimation (POISE)

We now introduce POISE, which leverages the policy model’s internal state signals for value estimation in RLVR. We first show that a lightweight probe can predict the value function, i.e., the expected verifier reward, directly from the policy model’s internal states (§ 3.1). We then integrate this probe into policy optimization to compute per-rollback advantages, yielding the full POISE algorithm without requiring a separate LLM-scale value model (§. 3.2).

3.1 Value Function Estimation from Policy Model Internal States

We introduce a probe designed to estimate baseline values directly from the policy model’s internal representations. Since the viability of this method hinges on the presence of such information, we additionally present preliminary empirical results demonstrating that these internal states inherently encode the necessary signals for accurate value estimation.

Probe prediction objective. The probe is trained to predict the prompt-level value under the current policy, defined as the expected verifier reward:

$$V^{\pi_\theta}(x) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[R(x, y)].$$

Since the ground-truth quantity is unknown, we instead sample K rollouts for each prompt x , $y^{(1)}, \dots, y^{(K)} \sim \pi_\theta(\cdot|x)$, and collect their verifier rewards $r^{(i)} = R(x, y^{(i)}) \in \{0, 1\}$. For the supervised example associated with rollout $y^{(i)}$, we use the leave-one-out Monte Carlo target as its gold value:

$$\widehat{V}_{-i}(x) = \frac{1}{K-1} \sum_{j \neq i} r^{(j)}.$$

By excluding $r^{(i)}$, $\widehat{V}_{-i}(x)$ remains conditionally independent of the input rollout $y^{(i)}$ given x , while still estimating $V^{\pi_\theta}(x)$ in expectation. This prevents the target from leaking the reward of the same rollout whose features are used by the probe.

Probe input features. As shown in Figure 2 (left), each supervised example for our probe is indexed by a prompt and one rollout, $(x, y^{(i)})$. For each pair, we construct the probe input from three complementary signals produced during the forward pass of the current policy π_θ . (All hidden-state features are extracted from a fixed layer ℓ , which we omit below for readability.) Let $H_{\theta,t}^{(i)}$ denote the residual-stream hidden state at token position t for $(x, y^{(i)})$, and let P and $R^{(i)}$ denote the final n prompt-token and reasoning-token positions, respectively.

First, we use the prompt-state feature $h_{\theta,p}^{(i)} = \text{Avg}_{t \in P} H_{\theta,t}^{(i)}$, motivated by evidence that prompt hidden states encode pre-generation estimates of difficulty and capability boundaries [42]. Second, we use the reasoning-state feature $h_{\theta,r}^{(i)} = \text{Avg}_{t \in R^{(i)}} H_{\theta,t}^{(i)}$, since trajectory-level hidden states can expose value-relevant information not available from the prompt states alone [39]. Third, we use token-level entropy statistics $u_\theta^{(i)}$ as lightweight uncertainty features [30]. The final probe input is

$$\phi_\theta^{(i)} = [h_{\theta,p}^{(i)}; h_{\theta,r}^{(i)}; u_\theta^{(i)}].$$

We ablate these input components in § 5 and the hidden-state extraction hyperparameters in § E.1. It is important to clarify that, while the input features include the generated reasoning, the estimator learns to predict the prompt-based value, rather than verifying its own reasoning, because the prediction target during training is the expected reward derived from other responses.

Probe implementation. We train lightweight regressors g_f to minimize the following loss.

$$\mathcal{L}_{\text{value}}(f) = \mathbb{E}_{x,i} \left[\left(g_f(\phi_\theta^{(i)}) - \widehat{V}_{-i}(x) \right)^2 \right].$$

Although our framework can theoretically support any regression architecture, we implement the probe using linear regression because its computational efficiency allows for fast, lightweight updates at each training step. We provide an ablation of probe designs in § E.2, and provide detailed implementations and hyperparameters in § B.3.

Preliminary experiment. Before using the probe for policy optimization, we first test whether the policy model’s internal states contain enough information to reliably estimate the prompt-level value. We construct a held-out value-prediction benchmark from reward-labeled rollouts of the DAPO-Math [37] dataset and compare two estimators trained on the same data: (1) a separate policy-scale critic model as a strong baseline (see § D.1 for details), and (2) our lightweight probe over the policy model’s internal state and entropy features. We evaluate both estimators on held-out prompts by comparing their predictions against the empirical Avg@8 reward.

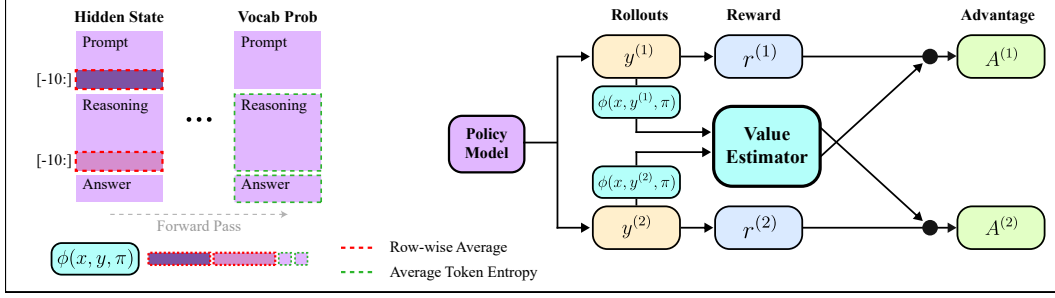


Figure 2: Overview of POISE. **Left:** Probe features $\phi(x, y, \pi)$ combine hidden states with token entropy. **Right:** The value estimator predicts each rollout’s baseline from the other rollout’s features.

Figure 1 shows that probes over the policy’s internal states achieve better held-out value prediction than the separate value model, despite adding only a lightweight regression head. This shows that the policy model’s own activations encode a compact signal about prompt difficulty and policy-specific uncertainty, which can be leveraged for value estimation at negligible cost.

3.2 Policy Optimization with Cross-Rollout Baselines

We now integrate the internal state probe into RL training as a value estimator, forming the full POISE algorithm (Figure 2 right).

Two rollouts per prompt. For each prompt $x \sim \mathcal{D}$ in the training batch we sample two independent rollouts from the current policy,

$$y^{(1)}, y^{(2)} \stackrel{\text{i.i.d.}}{\sim} \pi_{\theta_{\text{old}}}(\cdot | x), \quad (11)$$

and evaluate their verifiable rewards $R(x, y^{(1)})$ and $R(x, y^{(2)})$.

Cross-rollout baseline and advantage. The baseline for each rollout is predicted from the internal signals of the *other* rollout:

$$b^{(1)}(x) = g_f(\phi^{(2)}), \quad b^{(2)}(x) = g_f(\phi^{(1)}), \quad (12)$$

This yields the cross-rollout advantages

$$A^{(i)}(x) = R(x, y^{(i)}) - b^{(i)}(x), \quad i \in \{1, 2\}. \quad (13)$$

By construction, the baseline used to update $y^{(i)}$ depends only on the independently sampled rollout $y^{(j)}$, $j \neq i$, satisfying the conditional-independence condition in Eq. (6).

PPO-style policy update. We optimize the policy with a PPO-style clipped surrogate objective. Let $r_t^{(i)}(\theta) = \pi_{\theta}(y_t^{(i)} | x, y_{<t}^{(i)}) / \pi_{\theta_{\text{old}}}(y_t^{(i)} | x, y_{<t}^{(i)})$ be the importance ratio at token t of rollout i . The objective is

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim D, y^{(1)}, y^{(2)} \sim \pi_{\theta}(\cdot | x)} \left[\frac{1}{2} \sum_{i=1}^2 \frac{1}{|y^{(i)}|} \sum_{t=1}^{|y^{(i)}|} \min \left\{ r_t^{(i)}(\theta) A^{(i)}(x), \right. \right. \\ \left. \left. \text{clip} \left(r_t^{(i)}(\theta), 1 - \epsilon, 1 + \epsilon \right) A^{(i)}(x) \right\} \right]. \quad (14)$$

which we maximize with respect to θ over multiple inner epochs per batch.

Online estimator training with a trajectory buffer. The value estimator g_f is trained jointly with the policy on a sliding buffer of recent trajectories. At each step, for each prompt x with two independent rollouts $(y^{(1)}, y^{(2)})$, we construct value-estimator examples $\{(x, \phi(x, y^{(i)}), \hat{V}_{-i}(x))\}_{i=1,2}$,

Table 1: Performance comparison on olympiad level mathematical reasoning benchmarks. We report Avg@32 accuracy across various datasets. Our proposed internal state value estimation method (POISE) achieves competitive performance with the baseline models.

Model	Method	Benchmark							Avg
		AMC23	AMC24	AIME24	AIME25	AIME26	HMMT25	BRUMO25	
Qwen3-4B	<i>base</i>	0.422	0.319	0.263	0.196	0.244	0.129	0.217	0.258
	DAPO	0.876	0.607	0.490	0.475	0.457	0.267	0.384	0.508
	POISE (Ours)	0.891	0.592	0.469	0.437	0.443	0.280	0.387	0.500
Deepseek-Distill-Qwen-1.5B	<i>base</i>	0.169	0.078	0.067	0.067	0.104	0.021	0.042	0.078
	DAPO	0.697	0.447	0.254	0.219	0.198	0.065	0.191	0.296
	POISE (Ours)	0.694	0.446	0.270	0.234	0.213	0.066	0.198	0.303

where $\hat{V}_{-i}(x) = R(x, y^{(j)})$, $j \neq i$. We update f by minimizing a regression loss over the union of these newly generated examples and a buffer of examples from the most recent n steps. The buffer stabilizes the training signal under policy drift, while the joint update keeps g_f aligned with the value function of the evolving policy. Because g_f is a lightweight probe over signals already computed during the forward pass, this update is negligible in cost. The full procedure is summarized in Algorithm 1.

4 Experiments

4.1 Experimental Setup

Training. We instantiate our method on Qwen3-4B [35] and DeepSeek-R1-Distill-Qwen-1.5B [9], training on the English subset of DAPO-Math-17K [37] with batch sizes of 1024 and 512 on B200 GPUs. Rollouts are sampled with temperature 1.0 and top-p 1.0. Our main baseline is DAPO [37]: a state of the art, GRPO-based RLVR algorithm for mathematical reasoning. We adopt the implementation of Zheng et al. [41], which improves the efficiency of DAPO’s dynamic sampling. Full hyperparameters are provided in § B.4.

Evaluation. We evaluate our method on a suite of olympiad mathematical reasoning benchmarks: AMC23/24 [18], AIME24/25/26 [19], HMMT25 [10], and BRUMO25 [2]. For each benchmark, we report Avg@32, using temperature 0.6 and top-p 0.95 following common reasoning-model evaluation settings [9, 35]. By averaging over 32 sampled responses per problem, this protocol provides a reliable estimate of each model’s expected reasoning performance. We also compare training efficiency by analyzing the wall-clock time each method requires to achieve comparable reasoning performance. Detailed descriptions of each dataset, the full evaluation protocol are provided in § B.5.

4.2 Main Results on Math Reasoning Benchmarks

Table 1 reports the main results on olympiad-level mathematical reasoning benchmarks. For Qwen3-4B, POISE achieves an average Avg@32 score of 0.500, which is close to DAPO’s 0.508, while outperforming DAPO on AMC23, HMMT25, and BRUMO25. For Deepseek-Distill-Qwen-1.5B, POISE improves the average Avg@32 score from 0.296 to 0.303 over DAPO, with gains on AIME24, AIME25, AIME26, HMMT25, and BRUMO25. Across both model scales, these results indicate that POISE achieves performance comparable to a state-of-the-art RL algorithm while replacing group-relative baseline estimation with lightweight internal state value estimation. Detailed training dynamics are provided in C.

Training efficiency and stability. Figure 3 (left) shows that POISE requires substantially less wall-clock time per step than DAPO. The difference comes from how the two methods obtain usable advantage signals. In DAPO, the group-mean baseline becomes uninformative when all rollouts for a prompt receive the same reward, so dynamic sampling must first generate a full group of N rollouts to check whether the prompt yields nonzero advantages. When a group is degenerate, its rollouts are excluded from the final training batch for each step, forcing DAPO to sample additional groups until enough effective examples are collected. In contrast, POISE predicts the expected verifier reward as a

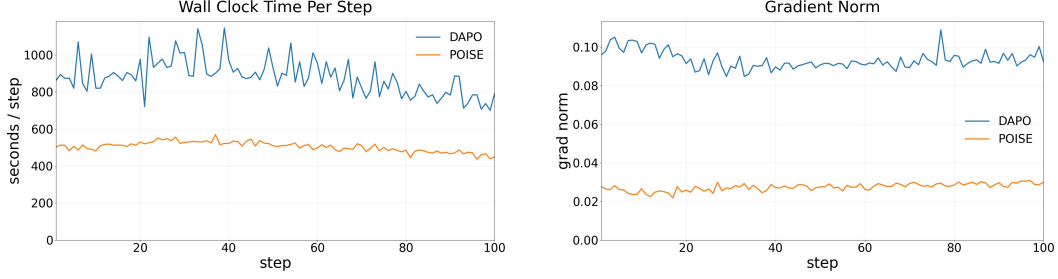


Figure 3: Comparison of training dynamics between POISE and DAPO on Deepseek-Distill-Qwen-1.5B. **Left:** wall-clock time per training step. **Right:** gradient norm at each step.

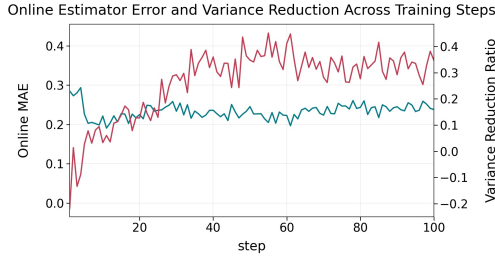


Figure 4: The green line reports the online MAE ($g_f(\phi), \bar{R}_t$), where $\bar{R}_t$ is the mean reward of the rollouts at step t . The red line reports the variance reduction ratio, $1 - \text{Var}(A)/\text{Var}(R)$, where $A = R - b$ is the advantage.

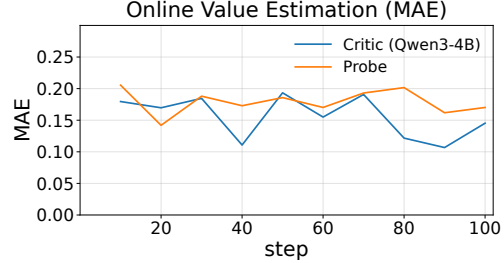


Figure 5: Comparison between our value estimator and a critic model in online settings. Our estimator remains well aligned with the evolving policy while using substantially less computation. For full results, refer to § D.2.

continuous value from internal state signals already produced during generation, thereby avoiding degeneration and saving a substantial amount of rollout compute. Concretely, in our setting, on DeepSeek-R1-Distill-Qwen-1.5B, reaching the same performance level takes roughly 24 hours of wall-clock time with DAPO on a single B200 GPU, compared to about 18 hours with POISE. We observe a similar trend on Qwen3-4B: POISE requires about 36 hours on two B200 GPUs, whereas DAPO takes 49 hours under the same hardware setting.

We further examine whether POISE leads to more stable optimization. DAPO and our method form the same gradient estimator through Eq. (9) and differ in how the baseline $b(x)$ is constructed. The expected squared norm of a policy-gradient estimator decomposes into the true gradient signal and an estimation-noise term. Since the true gradient depends only on the current policy and data, methods at similar training progress should have comparable signal magnitude; differences in gradient norm therefore mainly reflect differences in estimator noise. Under the same batch budget, POISE fits more distinct prompts than DAPO, which, in principle, reduces gradient variance according to § 2.3 and stabilizes training. Figure 3 (right) confirms this empirically. Our gradient-norm stays consistently lower than DAPO’s throughout training.

Training dynamics of value estimator. To evaluate whether the value estimator reliably tracks the evolving policy, we compute the online MAE (mean absolute error) between its predicted baseline values and the empirical mean reward of rollouts sampled from the current policy (Figure 4). The online MAE stays relatively stable across training, indicating that the estimator remains calibrated to the rewards produced by the current policy. Meanwhile, the variance reduction ratio remains around 30% after the initial phase, showing that the learned baseline reduces the reward variance by roughly one third when forming the advantage. Together, these results suggest that the online-trained estimator adapts to policy changes and provides a stable baseline throughout training.

Table 2: Performance of our estimator across multiple domains (Qwen3-4B). We compare against a separately trained critic and report MAE and Pearson correlation r . Full results are in Table 7.

Domain	Dataset	Critic		Ours	
		MAE ↓	r ↑	MAE ↓	r ↑
Math	DAPO-Math	0.262	0.676	0.141	0.870
	DeepScaleR	0.393	0.384	0.231	0.609
Coding	AceCoder	0.499	0.056	0.234	0.612
Tool	ToolDial	0.303	0.440	0.188	0.840
Instruction	IF-RLVR	0.350	0.150	0.195	0.642

Table 3: Ablation of estimator input features (Qwen3-4B). We report MAE and Pearson correlation r after training the estimator with only one feature type.

Input feature	MAE ↓	r ↑
only prompt hidden states	0.234	0.569
only reasoning hidden states	0.132	0.821
only mean entropy	0.152	0.780
only response length	0.251	0.494
Full estimator	0.126	0.838

5 Analysis of the Value Estimator

Comparison to an online policy-model scale critic. The previous training dynamics analysis evaluates whether our estimator serves as a stable baseline during policy optimization. Here, we additionally compare with a separately trained policy-model scale critic under policy drift. Using the Qwen3-4B training log from our main experiments, we train a critic model just like in § 3.1, but on accumulated reward-labeled rollouts and evaluate both estimators every 10 steps against empirical Avg@8 values from the corresponding actor checkpoint. As shown in Figures 5 and 10, the critic is slightly more accurate, likely due to its larger capacity and continual training on the expanding rollout log. Nevertheless, our estimator closely tracks the critic while requiring only a lightweight probe over features already produced by the policy forward pass.

Generalizability across domains and models. Next, we evaluate the estimator’s generalizability across multiple RLVR domains and policy models. For datasets, we include two mathematical reasoning tasks from DAPO-Math 17K [37] and DeepScaleR [17], coding tasks from AceCoder [38], tool-calling dialogues from ToolDial [26], and instruction-following tasks from IF-RLVR [22]. For policy models, we consider Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B/7B. For each domain-model pair, we train both our estimator and a critic model using the same data, and compare their predictions against the actual avg@8 scores of the target policy model (For detailed settings, refer to § D.3). We report representative results in Table 2 and the full results in Table 7. The estimator is competitive with and often more accurate than the critic model, which suggests that the policy’s hidden states expose a useful signal about whether the model is likely to produce a verifiably correct response. We therefore view our main experiments on math domain (§4.2) as a proof of concept for a more general RLVR mechanism: whenever verifiable feedback is available, a lightweight estimator can be trained online from the policy’s own internal states, providing a cheap value estimate without an auxiliary critic or large rollout groups.

Ablations. We ablate our value estimator along three axes: input features, hyperparameters for hidden state extraction, and probe architecture. We first ablate the input features used by our estimator. We use the same settings as the experiment in § 3.1 and train a value estimator with the following features. First, we evaluate the three core features of our method—prompt hidden states, reasoning hidden states, and vocabulary entropy. Following a prior work [5], we also evaluate response length. As shown in Table 3, trajectory-level features such as reasoning hidden states and mean entropy are influential in our value estimator’s performance: using either of the two retains much of the estimator’s accuracy. On the other hand, prompt hidden states show relatively low performance, and ablating the response length provides little or no improvement.

Next, we compare hyperparameter values used during hidden state extraction, such as layer index and mean pooling token size. As detailed in § E.1, our results show that mid-later layers are optimal, and the performance of the probe is not sensitive to token length.

Lastly, we compare our linear probe design with heavier models, such as Multi-Layer Perceptrons (MLPs). The results (Table 12) indicate that linear regression is just as effective as—and sometimes surpasses—the MLP probes. We attribute this to findings from prior work, which demonstrate that many semantic features are encoded as linear directions within the Transformer’s internal

representations [6, 21]. Consequently, a linear probe is naturally well-suited to extract these value signals efficiently without the need for additional model complexity or the risk of overfitting.

6 Related Work

Value Estimation in RL for LLM Reasoning. RL algorithms for LLMs reasoning differ mainly in how they estimate the baseline that reduces policy-gradient variance [20, 24, 25]. Recent work extends this along two axes. The first introduces explicit value models – either a generalist value prior for sparse rollouts [40] or a sequence-level value model that treats reasoning as a contextual bandit [31] – but incurs the substantial training, calibration, and deployment cost of an additional LLM-scale model. The second reduces rollout cost by non-uniform prompt sampling, via probabilistic informativeness-based filtering [41] or historical value tracking with global advantage normalization [34]; however, both rely on initial rollouts or per-prompt reward histories, which become prohibitive when rollout generation dominates RLVR cost [11]. In contrast, our method predicts the baseline at negligible cost by reusing the policy’s hidden states and generation signals already computed during the forward pass, eliminating both an auxiliary value model and pre-collected rollouts while preserving the variance-reduction benefit of value estimation.

Outcome-relevant Information in Hidden States. A growing body of work shows that the hidden states of large language models encode information relevant to assessing their outputs and task outcomes. Early studies on language model interpretability support this by showing that simple linear probes over internal representations can recover latent properties such as factuality [3], truthfulness [1, 15], confidence [43], and answer correctness. Recent studies extend this idea to reasoning models, where activations have been used to predict final correctness [4], identify capability boundaries between solvable and unsolvable prompts [42], estimate perceived difficulty and reasoning effort [39], and support self-verification or early stopping during generation [27].

Overall, prior work primarily uses hidden-state information as diagnostic signals or test-time control mechanisms. In contrast, we incorporate such signals directly into RL training by learning an online value estimator from the policy model’s own hidden states, yielding a cheap baseline without requiring an auxiliary LLM-scale critic or large rollout groups.

7 Conclusion

We introduce **POISE** (Policy Optimization with Internal State Value Estimation), which predicts rollout value from the policy’s internal states instead of relying on a group-mean baseline or a separate critic. To preserve unbiasedness, we couple this baseline with a cross-rollout construction. Our method achieves performance comparable to DAPO on mathematical reasoning benchmarks at a lower computational cost and more stable training. Finally, we show that our value estimator performs as well as a separate policy-scale value model and can generalize to other verifiable tasks.

8 Limitations and Future Work

Our experiments are conducted under a fixed compute budget, and while the trends we report are consistent across backbones and benchmarks, characterizing the behavior of internal state value estimation under substantially longer training horizons remains an interesting direction that we leave to future work with greater compute resources.

Several extensions naturally follow from our framework. First, our current estimator predicts value at the sequence level; extending it to token-level credit assignment would yield finer-grained advantages that more precisely reward the tokens responsible for a successful trajectory and penalize those that derail it, an effect known to be especially impactful for long reasoning trajectories [16, 30]. Second, internal state value estimates may also be useful beyond policy gradient optimization: for preference learning algorithms such as Direct Preference Optimization [23], they could inform the construction of response pairs by identifying rollouts with meaningfully different predicted values, potentially yielding more informative preference comparisons. Third, although we focused on mathematical reasoning as a controlled testbed, applying it to RL training for agentic reasoning and instruction-following tasks is a promising next step toward broader RLVR deployment.

Acknowledgments

We thank Haesung Pyun for helpful feedback and advice on improving the writing of this paper.

References

- [1] Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it’s lying. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 967–976, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.68. URL <https://aclanthology.org/2023.findings-emnlp.68/>.
- [2] Brown University Math Olympiad Team. BrUMO 2025: Brown University Mathematics Olympiad. <https://www.brumo.org>, 2025. Inaugural competition, held April 4–5, 2025, Brown University, Providence, RI.
- [3] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=ETKGuby0hcs>.
- [4] Iván Vicente Moreno Cencerrado, Arnau Padrés Masdemont, Anton Gonzalez Hawthorne, David Demitri Africa, and Lorenzo Pacchiardi. No answer needed: Predicting LLM answer accuracy from question-only linear probes, 2026. URL <https://openreview.net/forum?id=0hN25uxVab>.
- [5] Siddhartha Devic, Charlotte Peale, Arwen Bradley, Sinead Williamson, Preetum Nakkiran, and Aravind Gollakota. Trace length is a simple uncertainty signal in reasoning models. *arXiv preprint arXiv:2510.10409*, 2025.
- [6] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- [7] Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient llm post-training, 2025. URL <https://arxiv.org/abs/2510.01135>.
- [8] Evan Greensmith, Peter L. Bartlett, and Jonathan Baxter. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5:1471–1530, 2004.
- [9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li,

- Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- [10] HMMT Organization. HMMT February 2025: Harvard–MIT Mathematics Tournament. <https://www.hmmt.org/www/archive/282>, 2025. Individual round problems, February 2025, MIT, Cambridge, MA.
- [11] Baizhou Huang and Xiaojun Wan. PROS: Towards compute-efficient RLVR via rollout prefix reuse. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=lz1SRTcnUb>.
- [12] Aaron Jaech et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [14] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Øyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2025.
- [15] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 41451–41530. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/81b8390039b7302c909cb769f8b6cd93-Paper-Conference.pdf.
- [16] Zicheng Lin, Tian Liang, Jiahao Xu, Qiuzhi Lin, Xing Wang, Ruilin Luo, Chufan Shi, Siheng Li, Yujiu Yang, and Zhaopeng Tu. Critical tokens matter: Token-level contrastive estimation enhances llm’s reasoning capability, 2025. URL <https://arxiv.org/abs/2411.19943>.
- [17] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-01-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025. Notion Blog.
- [18] Mathematical Association of America. MAA American Mathematics Competitions (AMC). <https://maa.org/student-programs/amc/>, 2023–2026.
- [19] Mathematical Association of America. American Invitational Mathematics Examination (AIME). <https://maa.org/maa-invitational-competitions/>, 2024–2026.
- [20] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

- [21] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=UGpGkLzwpP>.
- [22] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL <https://openreview.net/forum?id=yfYgwjj5F8>.
- [23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. URL <https://arxiv.org/abs/2305.18290>.
- [24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [25] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [26] Jeonghoon Shim, Gyuhyeon Seo, Cheongsu Lim, and Yohan Jo. Tooldial: Multi-turn dialogue generation method for tool-augmented language models. *arXiv preprint arXiv:2503.00564*, 2025.
- [27] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=HvoG8SxggZ>.
- [28] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems (NIPS) 12*, pages 1057–1063. MIT Press, 2000. URL <https://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation>.
- [29] George Tucker, Surya Bhupatiraju, Shixiang Gu, Richard E. Turner, Zoubin Ghahramani, and Sergey Levine. The mirage of action-dependent baselines in reinforcement learning, 2018. URL <https://arxiv.org/abs/1802.10031>.
- [30] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=yfcpdY4gMP>.
- [31] Tianyi Wang, Yixia Li, Long Li, Yibiao Chen, Shaohan Huang, Yun Chen, Peng Li, Yang Liu, and Guanhua Chen. Sppo: Sequence-level ppo for long-horizon reasoning tasks, 2026. URL <https://arxiv.org/abs/2604.08865>.
- [32] Lex Weaver and Nigel Tao. The optimal reward baseline for gradient-based reinforcement learning. In *Proceedings of the 17th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 538–545, 2001.
- [33] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992. doi: 10.1007/BF00992696.
- [34] Zhongwen Xu and Zihan Ding. Single-stream policy optimization, 2025. URL <https://arxiv.org/abs/2509.13232>.

- [35] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [36] Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms, 2025. URL <https://arxiv.org/abs/2502.03373>.
- [37] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- [38] Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhui Chen. ACECODER: Acing coder RL via automated test-case synthesis. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12023–12040, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.587. URL <https://aclanthology.org/2025.acl-long.587/>.
- [39] Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they’re right: Probing hidden states for self-verification. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=06I0Av7683>.
- [40] Yi-Kai Zhang, Yueqing Sun, Hongyan Hao, Qi Gu, Xunliang Cai, De-Chuan Zhan, and Han-Jia Ye. $v_{0.5}$: Generalist value model as a prior for sparse rl rollouts, 2026. URL <https://arxiv.org/abs/2603.10848>.
- [41] Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for llm reasoning via selective rollouts, 2025. URL <https://arxiv.org/abs/2506.02177>.
- [42] Yubo Zhu, Dongrui Liu, Zecheng Lin, Wei Tong, Sheng Zhong, and Jing Shao. The LLM already knows: Estimating LLM-perceived question difficulty via hidden representations. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1160–1176, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.61. URL <https://aclanthology.org/2025.emnlp-main.61/>.
- [43] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xu Wang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A Theoretical Proofs

A.1 Proof of Proposition 1

Proof. Define:

$$\mu(x) = \mathbb{E}[Z(x, y) \mid x], \quad \Sigma_w = \mathbb{E}_x[\text{Cov}(Z(x, y) \mid x)], \quad \Sigma_b = \text{Cov}_x(\mu(x)).$$

Let $G_i = \frac{1}{m} \sum_{j=1}^m Z(x_i, G_{ij})$. Since completions within a prompt are conditionally independent given x_i ,

$$\text{Cov}(G_i \mid x_i) = \frac{1}{m} \text{Cov}(Z(x_i, y) \mid x_i).$$

Applying the law of total covariance to G_i :

$$\text{Cov}(G_i) = \mathbb{E}_x[\text{Cov}(G_i \mid x_i)] + \text{Cov}_x(\mathbb{E}[G_i \mid x_i]) = \frac{1}{m} \Sigma_w + \Sigma_b.$$

Since prompts are sampled independently, $\text{Cov}(\hat{g}) = \frac{1}{n} \text{Cov}(G_i)$. Substituting $n = B/m$:

$$\text{Cov}(\hat{g}) = \frac{m}{B} \left(\frac{1}{m} \Sigma_w + \Sigma_b \right) = \frac{1}{B} \Sigma_w + \frac{m}{B} \Sigma_b. \quad \square$$

A.2 Proof of Corollary 1

Proof. From Proposition 1, for any allocation (n, m) with $nm = B$,

$$\text{Cov}(\hat{g}) = \frac{1}{B} \Sigma_w + \frac{m}{B} \Sigma_b.$$

Consider any two allocations $m_1 < m_2$ with the same budget B . Their difference is

$$\text{Cov}(\hat{g}_{m_2}) - \text{Cov}(\hat{g}_{m_1}) = \frac{m_2 - m_1}{B} \Sigma_b.$$

Since $\Sigma_b = \text{Cov}_x(\mu(x))$ is a covariance matrix, it is positive semidefinite, so for any vector v ,

$$v^\top (\text{Cov}(\hat{g}_{m_2}) - \text{Cov}(\hat{g}_{m_1})) v = \frac{m_2 - m_1}{B} v^\top \Sigma_b v \geq 0.$$

Therefore $\text{Cov}(\hat{g}_{m_2}) \succeq \text{Cov}(\hat{g}_{m_1})$ for any $m_2 > m_1$, meaning the variance of $\hat{g}$ in every gradient direction is non-decreasing in m . The minimum is thus attained at $m = 1$, giving $n = B$. $\square$

B Implementation Details

B.1 Pseudocode for POISE

Algorithm 1 POISE

Require: Prompt distribution $\mathcal{D}$; initial policy π_{θ_0} ; initial value estimator g_{f_0} ; prompt batch size M ; value-buffer size n ; PPO clip ϵ .

- 1: Initialize $\theta \leftarrow \theta_0$, $f \leftarrow f_0$, value buffer $\mathcal{B}_V \leftarrow \emptyset$.
- 2: **for** step = 1, 2, ..., T **do**
- 3: $\theta_{\text{old}} \leftarrow \theta$
- 4: Sample a mini-batch of prompts $\{x_b\}_{b=1}^M \sim \mathcal{D}$.
- 5: $\mathcal{R} \leftarrow \emptyset$, $\mathcal{S}_V \leftarrow \emptyset$
- 6: **for** each prompt x_b **do**
- 7: Sample $y_b^{(1)}, y_b^{(2)} \stackrel{\text{i.i.d.}}{\sim} \pi_{\theta_{\text{old}}}(\cdot | x_b)$.
- 8: Extract internal state features $\phi_b^{(i)} \leftarrow \phi_{\theta_{\text{old}}}(x_b, y_b^{(i)})$ for $i \in \{1, 2\}$ via forward hooks.
- 9: Compute rewards $r_b^{(i)} \leftarrow R(x_b, y_b^{(i)})$ for $i \in \{1, 2\}$.
- 10: Compute cross-rollout baselines:
 $b_b^{(1)} \leftarrow g_f(\phi_b^{(2)}), \quad b_b^{(2)} \leftarrow g_f(\phi_b^{(1)})$.
- 11: Compute advantages $A_b^{(i)} \leftarrow r_b^{(i)} - b_b^{(i)}$ for $i \in \{1, 2\}$.
- 12: Add policy-update examples:
 $\mathcal{R} \leftarrow \mathcal{R} \cup \{(x_b, y_b^{(i)}, A_b^{(i)})\}_{i=1,2}$.
- 13: Add value-estimator examples:
 $\mathcal{S}_V \leftarrow \mathcal{S}_V \cup \{(\phi_b^{(1)}, r_b^{(2)}), (\phi_b^{(2)}, r_b^{(1)})\}$.
- 14: **end for**
- 15: Update θ by maximizing the PPO objective in Eq. (14) on $\mathcal{R}$.
- 16: Update f by minimizing

$$\sum_{(\phi, \tilde{r}) \in \mathcal{B}_V \cup \mathcal{S}_V} (g_f(\phi) - \tilde{r})^2.$$
- 17: Append $\mathcal{S}_V$ to $\mathcal{B}_V$; evict examples older than n steps.
- 18: **end for**
- 19: **return** π_θ, g_f .

B.2 Baseline Algorithm

We adopt DAPO [37] as our baseline RL algorithm. For each prompt $q \sim \mathcal{D}$, a group of G responses $\{o^{(i)}\}_{i=1}^G$ is sampled from the old policy $\pi_{\theta_{\text{old}}}$, and the following objective is optimized:

$$\begin{aligned} \mathcal{J}_{\text{DAPO}}(\theta) = & \mathbb{E}_{(q, a) \sim \mathcal{D}, \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)} \\ & \left[\frac{1}{\sum_{i=1}^G |o^{(i)}|} \sum_{i=1}^G \sum_{t=1}^{|o^{(i)}|} \min \left(r_t^{(i)}(\theta) \hat{A}_t^{(i)}, \text{clip}(r_t^{(i)}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_t^{(i)} \right) \right], \\ & \text{s.t. } 0 < |\{o^{(i)} \mid \text{is_equivalent}(a, o^{(i)})\}| < G, \end{aligned} \quad (15)$$

where the importance ratio and the group-relative advantage are

$$r_t^{(i)}(\theta) = \frac{\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})}, \quad \hat{A}_t^{(i)} = \frac{R^{(i)} - \text{mean}(\{R^{(j)}\}_{j=1}^G)}{\text{std}(\{R^{(j)}\}_{j=1}^G)}.$$

DAPO augments GRPO with four key techniques: (i) **Clip-Higher** decouples the lower and upper clipping bounds ($\varepsilon_{\text{high}} > \varepsilon_{\text{low}}$), giving low-probability tokens more room to be promoted and mitigating entropy collapse; (ii) **Dynamic Sampling** filters out prompts whose responses are all correct or all incorrect, ensuring that every batch yields effective gradients; (iii) **Token-Level Policy Gradient Loss** normalizes by the total token count $\sum_i |o^{(i)}|$ rather than per-sequence, so longer

responses contribute proportionally to the loss; (iv) **Overlong Reward Shaping** applies a length-aware penalty to truncated samples to reduce reward noise.

Reward. Unlike the original DAPO, we drop the length penalty and use a purely binary reward based solely on correctness for simplicity:

$$R^{(i)} = \begin{cases} 1, & \text{if } o^{(i)} \text{ is correct,} \\ 0, & \text{otherwise.} \end{cases}$$

B.3 Value Estimator

Hidden states used for estimator training are collected from the model’s teacher-forced log probability forward pass, which is already required by our policy optimization algorithm and therefore does not need additional computation. Specifically, the implementation registers a forward hook on one transformer layer during this log probability computation, and pools prompt hidden states and reasoning hidden states separately.

Table 4 lists the hyperparameters used by the hidden-state value estimator. The prompt and reasoning hidden-state features are obtained by mean-pooling the last 10 prompt tokens and the last 10 reasoning tokens, respectively. For a pair of rollouts $(y^{(1)}, y^{(2)})$ from the same prompt, the supervised target for the feature row of $y^{(i)}$ is the paired rollout reward $\tilde{R}^{(i)} = R(x, y^{(j)}), j \neq i$.

Component	Setting
Hidden layer	Qwen3-4B: 19 DeepSeek-R1-Distill-Qwen-1.5B: 19
Prompt hidden state pooling	Last 10 prompt token mean
Reasoning hidden state pooling	Last 10 reasoning token mean
Scalar features	3 entropy statistics
Final estimator input dimension	$2d_{\text{model}} + 3$ Qwen3-4B: 5123 DeepSeek-R1-Distill-Qwen-1.5B: 3075
Regressor	StandardScaler + Ridge
Ridge penalty	$\alpha = 0.01$
Random seed	42
Prediction range	Clipped to $[0, 1]$

Table 4: Hyperparameters for the hidden-state value estimator. Here, d_{model} denotes the hidden-state dimension of each policy backbone.

B.4 Training details

We instantiate our method on two reasoning model backbones, Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B, with all training implemented in the `verl` library and the maximum response length capped at 8,192 tokens. As training data, we use DAPO-Math-17K [37] after filtering out Chinese-language prompts, yielding an English-only mathematical reasoning corpus. We use a training batch size of 1,024 prompts for Qwen3-4B and 512 prompts for DeepSeek-R1-Distill-Qwen-1.5B, and sample rollouts with temperature 1.0 and top-p 1.0. All experiments are run on B200 GPUs. Our main baseline is DAPO [37], for which we adopt the implementation and hyperparameters of Zheng et al. [41], which improves the efficiency of DAPO’s dynamic sampling. DAPO baseline draws 8 rollouts per prompt while our method draws a pair of rollouts per prompt and forms its baseline through the cross-rollout value probe described in § 3. Detailed hyperparameter values are reported in Table 5 and 6.

B.5 Evaluation Protocol

The following section details the benchmarks used in our experiments and our evaluation protocol.

Benchmarks.

Hyperparameter	Value
algorithm	POISE
training steps	120
train batch size	512
mini batch size	16
max prompt length	2048
max response length	8192
learning rate	1×10^{-6}
clip ratio (low / high)	0.2 / 0.28
entropy coefficient	0
use KL loss	False
sampling temperature	1.0
sampling top-p	1.0
samples per prompt	2
max batched tokens	10240

Table 5: Key training hyperparameters used in POISE.

Hyperparameter	Value
algorithm	DAPO
training steps	100
train batch size	128
mini batch size	16
p_{easy}	0.5
p_{hard}	0.5
target zero variance	0.25
default br size	192
GRESO min p	0.05
GRESO max p	0.95
β	1.25
max prompt length	2048
max response length	8192
learning rate	1×10^{-6}
clip ratio (low / high)	0.2 / 0.28
entropy coefficient	0
use KL loss	False
sampling temperature	1.0
sampling top-p	1.0
samples per prompt	8
max batched tokens	10240

Table 6: Key training hyperparameters used in DAPO with efficient dynamic sampling [41].

- **AMC23** and **AMC24** are problem sets from the American Mathematics Competition (AMC) [18], a series of contests for high school students that test problem-solving ability across a wide range of topics. We employ the 2023 and 2024 editions, consisting of 40 problems each (80 problems in total).
- **AIME24**, **AIME25**, and **AIME26** are problem sets from the American Invitational Mathematics Examination (AIME) [19], a prestigious competition featuring challenging problems that require sophisticated mathematical reasoning. We employ the 2024, 2025, and 2026 editions, consisting of 30 problems each (90 problems in total).
- **HMMT25** consists of problems from the February 2025 Harvard–MIT Mathematics Tournament (HMMT) [10], one of the most demanding high-school mathematics competitions in the United States. We use the individual-round problems, consisting of 30 problems.

- **BRUMO25** is the 2025 edition of the Brown University Mathematics Olympiad (BRUMO) [2], an annual olympiad-level competition for advanced high-school students. We use the official 2025 problem set, consisting of 30 problems.

Evaluation Protocol For all benchmarks above, we follow the officially recommended decoding setting with temperature 0.6 and top- p 0.95, and set the maximum response length to 8192 tokens the same as the training setting. All inferences are performed with the vLLM library [13] on a single node equipped with two NVIDIA B200 GPUs. For each problem, we independently sample 32 completions.

We define a binary reward $r^{(ij)} \in \{0, 1\}$ equal to 1 if the j -th response to problem i yields a correct final answer and 0 otherwise; the same reward function is used during RL training and evaluation. Given a test set with M problems, we report:

- **avg@ k** : the expected per-sample correctness,

$$\text{avg@}k = \frac{1}{M} \sum_{i=1}^M \frac{1}{k} \sum_{j=1}^k r^{(ij)}.$$

Throughout the paper we use $k = 32$ for both metrics.

B.6 Prompt Template

We use the following single-turn prompt template, following DAPO [37], for all mathematical reasoning tasks. The placeholder {problem} is replaced with the problem statement at training and inference time.

Prompt Template

Solve the following math problem step by step. The last line of your response should be of the form Answer: \$Answer (without quotes) where \$Answer is the answer to the problem.

{problem}

Remember to put your answer on its own line after “Answer:”.

C Training Dynamics

We provide additional training dynamics for POISE on Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B. In the main text, we report aggregate evidence that the internal state value estimator remains stable during training. Here, we expand the analysis by tracking five quantities over policy updates: the batch reward, the predicted value, token-level entropy, the value-estimation error against the online target, and the advantage variance ratio.

For each training step t , let R_t denote the verifier reward of the sampled rollouts, $b_t = g_f(\phi_t)$ the predicted baseline, and $A_t = R_t - b_t$ the resulting advantage. We report the batch reward $\bar{R}_t = \mathbb{E}_{\text{batch}}[R_t]$, the mean predicted value $\bar{b}_t = \mathbb{E}_{\text{batch}}[b_t]$, the online target error $\text{MAE}_t = \mathbb{E}_{\text{batch}}[|b_t - \hat{V}_t(x)|]$, and the advantage variance ratio $\rho_t = \text{Var}_{\text{batch}}(A_t) / \text{Var}_{\text{batch}}(R_t)$. Here, $\hat{V}_t(x)$ is the empirical online target estimated from rollouts sampled from the current checkpoint policy. The ratio ρ_t measures how much variance remains after subtracting the learned baseline; values below one indicate that the estimator reduces variance relative to using the raw reward.

Qwen3-4B. Figure 7 shows the full dynamics for Qwen3-4B. The batch reward increases rapidly during the first phase of training, from roughly 0.35 to above 0.65, and then stabilizes around 0.70. The predicted value follows the same broad trend, rising from approximately 0.30 to the 0.70-0.75 range. This agreement suggests that the online estimator tracks the changing reward scale as the policy improves, rather than remaining calibrated only to the initial policy.

The online target MAE decreases sharply in early training and then remains in a stable range. This is important because the value function itself is nonstationary during RL: as the actor improves, the target expected reward for each prompt also changes. The stability of the MAE therefore indicates that the sliding-buffer update is sufficient for tracking the evolving policy. The advantage variance ratio remains below one for most of training and stabilizes after the initial phase, showing that the learned baseline continues to reduce variance after the policy has moved away from its initialization. Finally, entropy increases rather than collapsing, suggesting that POISE does not obtain its gains by prematurely concentrating the policy distribution.



Figure 6: The green line reports the online $\text{MAE}(g_f(\phi), \bar{R}_t)$, where $\bar{R}_t$ is the mean reward of the rollouts at step t . The red line reports the variance reduction ratio, $1 - \text{Var}(A) / \text{Var}(R)$, where $A = R - b$ is the advantage.

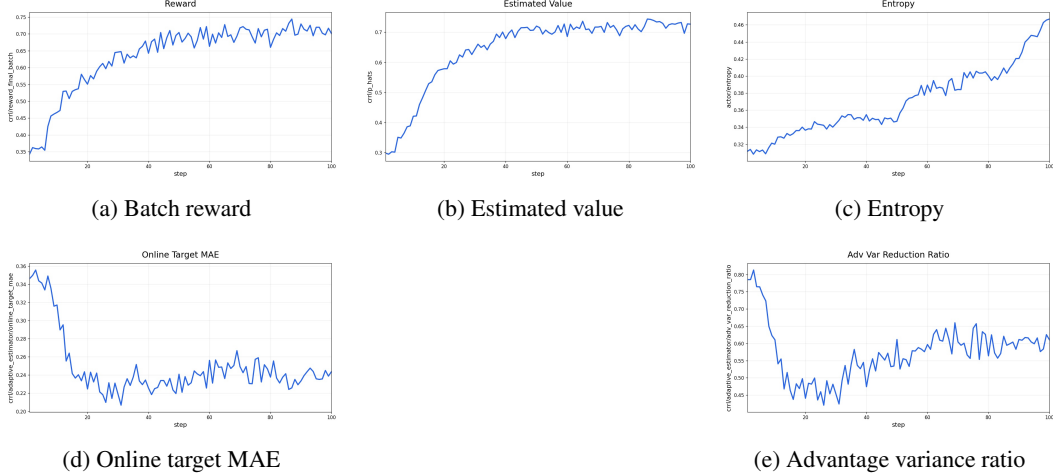


Figure 7: Training dynamics of POISE on Qwen3-4B. The reward and predicted value increase together as the policy improves. The online target MAE remains stable after the early phase, and the advantage variance ratio stays below one for most of training, indicating that the learned baseline reduces reward variance when forming advantages.

DeepSeek-R1-Distill-Qwen-1.5B. Figure 8 reports the same diagnostics for DeepSeek-R1-Distill-Qwen-1.5B. The smaller backbone shows noisier dynamics, which is expected because its reward distribution is lower and more variable. Even in this setting, the reward improves throughout training, increasing from roughly 0.15-0.20 in the initial phase to around 0.45-0.50 by the end of training. The estimated value also increases over the course of training, although it exhibits an early drop before recovering. This transient mismatch likely reflects the difficulty of fitting the online estimator in the first few updates, when the buffer contains limited data and the policy distribution changes quickly.

After this initial phase, the estimator becomes more stable. The online target MAE remains bounded and does not diverge as training proceeds, indicating that the estimator continues to track the current policy despite the nonstationary target. The advantage variance ratio also falls below one after the early updates and remains there for most of the run, showing that the learned baseline provides variance reduction even for the smaller and noisier policy. Entropy rises during the early-to-middle phase and then fluctuates mildly, suggesting that POISE maintains policy stochasticity rather than driving immediate entropy collapse.

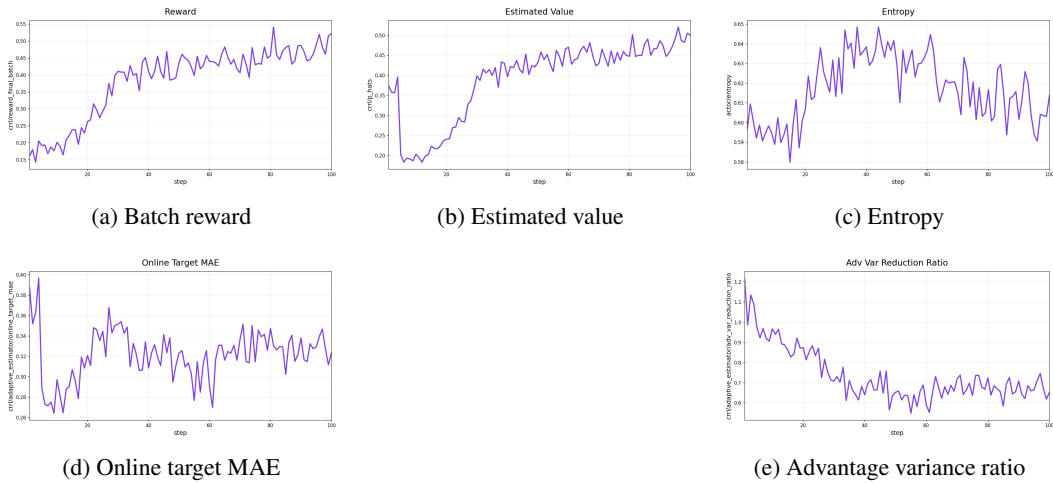


Figure 8: Training dynamics of POISE on DeepSeek-R1-Distill-Qwen-1.5B. Although the smaller model exhibits noisier dynamics, the reward improves steadily and the estimated value follows the increasing reward scale after the initial phase. The online target MAE remains bounded, and the advantage variance ratio stays below one for most of training, indicating effective variance reduction.

D Comparison to Policy-model Scale Critic Model

D.1 Critic Model Implementation

For the critic baseline in the value-prediction comparison, we implement a sequence-level scalar critic following SPPO [31]. Unlike the token-level PPO critic, which predicts a value $V(s_t)$ for every intermediate token state and relies on GAE to propagate sparse terminal rewards, the SPPO critic treats long-form reasoning as a sequence-level contextual bandit. The prompt x is the context, the full response y is the action, and the verifier reward $R(x, y)$ is an outcome-level binary reward. Accordingly, the critic predicts a single prompt-level scalar value

$$V_\phi(x) \approx \mathbb{E}_{y \sim \pi(\cdot|x)}[R(x, y)], \quad (16)$$

which can be interpreted as the policy’s estimated probability of solving the prompt.

Architecturally, the critic is initialized from the corresponding policy backbone and augmented with a scalar value head. We feed the chat-formatted prompt into the model, collect the hidden state at the final non-padding token, and apply a linear head to produce a scalar logit:

$$z_\phi(x) = w^\top h_\phi(x), \quad V_\phi(x) = \sigma(z_\phi(x)). \quad (17)$$

In the POISE comparison, the critic is trained only as an analysis baseline and is never used to compute POISE advantages. Given reward-labeled rollouts for a prompt, we aggregate independently sampled verifier rewards into an empirical prompt value, e.g. Avg@ K :

$$\hat{V}(x) = \frac{1}{K} \sum_{j=1}^K R(x, y^{(j)}). \quad (18)$$

The critic is then optimized to predict this prompt-level target. Specifically, we use the Bernoulli objective,

$$\mathcal{L}_{\text{BCE}}(\phi) = \text{BCEWithLogitsLoss}(z_\phi(x), R). \quad (19)$$

At evaluation time, we compare critic predictions against held-out empirical prompt values using mean absolute error (MAE) and Pearson correlation.

D.2 Comparison with an Online Policy-model Scale Critic

§ 3.1 evaluated value prediction on a fixed policy distribution. We further test whether the internal state estimator remains competitive in the online setting, where the policy changes over the course of RL training. At every 10 training steps, we take the corresponding actor checkpoint, sample eight responses for each prompt used in the step, and use the empirical Avg@8 score as the target prompt value. We then compare two estimators against this target: a separately trained policy-scale critic and our lightweight internal state estimator.

Figures 9 and 10 show the result up to step 100 for Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B. We report both Pearson correlation and mean absolute error (MAE) to the Avg@8 target.

For both Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B, the internal state estimator closely tracks the critic throughout training. While a separate critic can be slightly more accurate because it is an LLM-scale model trained on accumulated rollout data, our estimator remains close to the critic across both model scales while using only hidden-state and entropy features already produced by the policy forward pass. This supports the central motivation of POISE: internal states provide a practical value signal that tracks the evolving policy without maintaining an additional critic model.

D.3 Comparisons on Multiple Domains and Models

We evaluate whether our internal-state value estimator generalizes to different verifiable-reward domains and policy backbones. We consider five datasets spanning math reasoning, coding, tool use, and instruction following: DAPO-Math [37], DeepScaleR [17], AceCoder [38], ToolDial [26], and IF-RLVR [22]. We also evaluate three policy backbones: Qwen3-4B, DeepSeek-R1-Distill-Qwen-1.5B, and DeepSeek-R1-Distill-Qwen-7B.

For each model–dataset pair, we train the internal-state estimator and a separately trained critic on the same reward-labeled rollout data. To make the comparison consistent across settings, we use 4,096

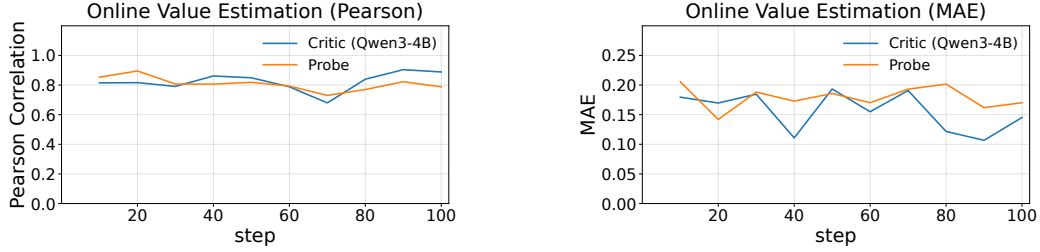


Figure 9: Online value prediction for Qwen3-4B. The target at each checkpoint is the empirical Avg@8 score from the corresponding actor checkpoint.

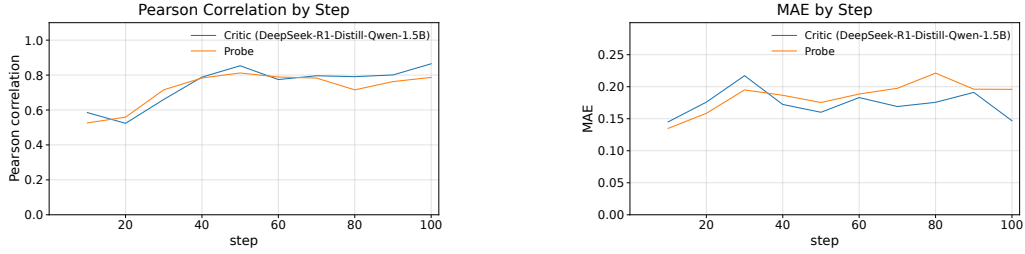


Figure 10: Online value prediction for DeepSeek-R1-Distill-Qwen-1.5B. The target at each checkpoint is the empirical Avg@8 score from the corresponding actor checkpoint.

training examples for each estimator, matching the number of examples stored in the value-estimator buffer used by the POISE algorithm. We then evaluate both estimators against empirical Avg@8 values on held-out prompts. For evaluation, we use 1,024 held-out prompts for all datasets except ToolDial, for which we use 13,492 held-out examples.

Table 7 reports the results. On Qwen3-4B, the internal state estimator consistently outperforms the critic across all five domains, reducing MAE and improving Pearson correlation in every setting. The gains are especially large outside the original math setting: on AceCoder, Pearson correlation increases from 0.056 to 0.612, and on IF-RLVR it increases from 0.150 to 0.642. These results indicate that the estimator is not merely exploiting dataset-specific artifacts from DAPO-Math; it can recover value-relevant information from policy activations across substantially different forms of verifiable feedback.

The trend is more mixed but still favorable on DeepSeek-R1-Distill-Qwen-1.5B. The internal state estimator improves over the critic on DeepScaleR and ToolDial, while the critic is stronger on AceCoder and slightly better in MAE on IF-RLVR. Even in these weaker cases, the estimator remains competitive in correlation despite using only lightweight hidden-state and entropy features. For the completed DeepSeek-R1-Distill-Qwen-7B settings, the estimator again improves over the critic on both DAPO-Math and ToolDial, suggesting that the signal persists at a larger model scale.

Overall, these results support the broader applicability of POISE beyond the mathematical-reasoning training experiments.

Table 7: Full value-prediction results across policy backbones and verifiable-reward domains. We compare our internal state estimator against a separately trained critic and report MAE and Pearson correlation r .

Policy model	Domain	Dataset	Critic		internal state estimator	
			MAE ↓	r ↑	MAE ↓	r ↑
Qwen3-4B	Math reasoning	DAPO-Math	0.262	0.676	0.141	0.870
		DeepScaleR	0.393	0.384	0.231	0.609
	Coding	AceCoder	0.499	0.056	0.234	0.612
	Tool calling	ToolDial	0.303	0.440	0.188	0.840
	Instruction following	IF-RLVR	0.350	0.150	0.195	0.642
DeepSeek-R1-Distill-Qwen-1.5B	Math reasoning	DAPO-Math	0.127	0.723	0.123	0.834
		DeepScaleR	0.251	0.586	0.151	0.829
	Coding	AceCoder	0.135	0.706	0.196	0.580
	Tool calling	ToolDial	0.201	0.337	0.122	0.672
	Instruction following	IF-RLVR	0.101	0.545	0.171	0.557
DeepSeek-R1-Distill-Qwen-7B	Math reasoning	DAPO-Math	0.300	0.441	0.191	0.784
		DeepScaleR	0.270	0.457	0.164	0.814
	Coding	AceCoder	0.229	0.691	0.173	0.804
	Tool calling	ToolDial	0.194	0.663	0.153	0.842
	Instruction following	IF-RLVR	0.155	0.716	0.191	0.596

E Ablations

E.1 Ablations of Hyperparameters During Hidden State Extraction

We next ablate the hidden-state extraction hyperparameters of the Qwen3-4B estimator. Unless otherwise specified, we keep the estimator architecture fixed as StandardScaler followed by ridge regression, with prompt hidden states projected to 32 dimensions and trajectory hidden states projected to 256 dimensions using PCA. The scalar entropy features are kept fixed across all runs.

Layer index. We first fix the pooling window to the last 10 tokens and sweep the transformer layer used to extract both prompt and trajectory hidden states. For Qwen3-4B, Table 8 shows that performance is already strong in early layers but improves toward the middle of the network. The best Pearson correlation is obtained at layer 19, while layer 33 gives the lowest MAE. We use layer 19 as the default for the main experiments because it gives the strongest correlation with verifier value while remaining near-optimal in MAE.

Table 8: Layer ablation for Qwen3-4B with last-10-token mean pooling.

Layer	MAE ↓	Pearson r ↑
1	0.128	0.807
3	0.126	0.821
5	0.129	0.816
7	0.129	0.820
9	0.131	0.814
11	0.131	0.819
13	0.130	0.823
15	0.128	0.828
17	0.124	0.831
19	0.123	0.834
21	0.128	0.830
23	0.130	0.824
25	0.132	0.817
27	0.125	0.827
29	0.125	0.830
31	0.126	0.824
33	0.120	0.831
35	0.123	0.827

For DeepSeek-R1-Distill-Qwen-1.5B, we repeat the same odd-layer sweep using prompt hidden states and reasoning-trajectory hidden states, both mean-pooled over the last 10 tokens. As shown in Table 9, the strongest validation performance appears in the earliest layer. However, the differences across layers are small in absolute MAE, and layer 19 remains close to the best setting while using the same extraction configuration as Qwen3-4B. We therefore use layer 19 for both backbones in the main POISE experiments to avoid model-specific layer tuning and keep the estimator implementation consistent across models.

Pooling window. We also vary the number of final tokens used for mean pooling while fixing the layer to 19. As shown in Table 10, last-10 and last-15 pooling perform almost identically, while last-5 pooling is weaker. We choose last-10 pooling as the default because it achieves the best Pearson correlation while using a shorter extraction window than last-15.

Table 10: Pooling-window ablation for Qwen3-4B at layer 19.

Pooling window	MAE ↓	Pearson r ↑
Last 5	0.127	0.822
Last 10	0.124	0.834
Last 15	0.124	0.833

Table 9: Layer ablation for DeepSeek-R1-Distill-Qwen-1.5B with last-10-token mean pooling.

Layer	MAE ↓	Pearson r ↑
1	0.134	0.425
3	0.135	0.405
5	0.135	0.418
7	0.136	0.403
9	0.137	0.369
11	0.141	0.332
13	0.139	0.339
15	0.137	0.380
17	0.136	0.382
19	0.135	0.395
21	0.135	0.378
23	0.135	0.332
25	0.139	0.290
27	0.139	0.292

For DeepSeek-R1-Distill-Qwen-1.5B, we fix the layer to 1, which was selected by the layer sweep, and repeat the pooling-window ablation in Table 11. The three pooling windows are close, but last-10 pooling gives the best MAE and Pearson correlation.

Table 11: Pooling-window ablation for DeepSeek-R1-Distill-Qwen-1.5B at layer 1.

Pooling window	MAE ↓	Pearson r ↑
Last 5	0.1342	0.4230
Last 10	0.1339	0.4247
Last 15	0.1342	0.4200

Overall, these ablations support the robustness of the hidden-state extraction choice used in the main POISE experiments. The estimator is not overly sensitive to the exact pooling window, and while the best layer can be model-dependent, a shared layer-19 configuration provides competitive performance for both backbones. We therefore use layer 19 with last-10-token mean pooling as the default extraction setting in the main experiments.

E.2 Ablations on Probe Designs

We ablate the architecture of the lightweight value estimator while keeping the input representation fixed. All models use the Qwen3-4B features from layer 19 with last-10-token mean pooling, including prompt hidden states, trajectory hidden states, and entropy statistics. The train/test split are identical to those used in § E.1. We compare the default linear ridge regressor against multi-layer perceptrons (MLPs) with different depths and widths. MLPs are trained with AdamW, ReLU activations, dropout, and early stopping on a held-out validation split from the training set.

Table 12 shows that increasing model capacity can slightly reduce MAE: the best MLP, a 3-layer network with width 1024, improves MAE from 0.124 to 0.117. However, this improvement does not translate into better rank alignment with the target values. The linear ridge probe achieves the highest Pearson correlation, 0.834, while all MLP variants obtain lower correlation. We therefore use the linear estimator in the main method: it is cheaper to fit online, has fewer hyperparameters, and provides the most reliable correlation with verifier value, which is important for forming stable advantages.

Table 12: Probe architecture ablation for Qwen3-4B at layer 19 with last-10-token mean pooling. MLP names indicate depth $\times$ hidden width. Lower MAE and higher Pearson correlation are better.

Probe architecture	MAE $\downarrow$	Pearson r $\uparrow$
Linear ridge	0.124	0.834
MLP 1 $\times$ 512	0.154	0.778
MLP 3 $\times$ 512	0.123	0.780
MLP 5 $\times$ 512	0.128	0.765
MLP 7 $\times$ 512	0.124	0.763
MLP 9 $\times$ 512	0.133	0.814
MLP 3 $\times$ 1024	0.117	0.801
MLP 5 $\times$ 1024	0.128	0.754

Overall, this result suggests that the value-relevant signal in the policy’s internal states is largely linearly accessible. Larger nonlinear probes can improve absolute calibration in some cases, but they are less stable in correlation and introduce additional online-training cost. This supports our choice of a linear probe as the default estimator for POISE.